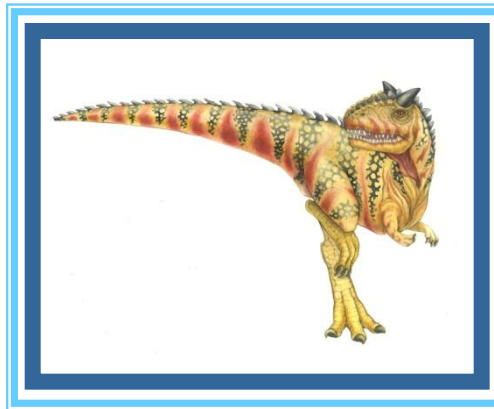


Chapter 11:

File-System Interface





Chapter 11: File-System Interface

- File Concept
- Access Methods
- Disk and Directory Structure
- File-System Mounting





File Concept

- the operating system provides a uniform logical view of information storage.
- The operating system abstracts from the physical properties of its storage devices to define a logical storage unit, the *file*.
- Files are mapped by the operating system onto physical devices.
- These storage devices are usually nonvolatile, so the contents are persistent through power failures and system reboots.
- **A file is a named collection of related information** that is recorded on secondary storage.
- file is the smallest allotment of logical secondary storage; that is, data cannot be written to secondary storage unless they are within a file.
- Commonly, files represent programs (both source and object forms) and data.
- Data files may be numeric, alphabetic, alphanumeric, or binary.





File Attributes

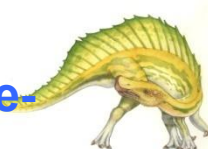
- ❑ **Name** – only information kept in human-readable form
 - ❑ **Identifier** – unique tag (number) identifies file within file system
 - ❑ **Type** – needed for systems that support different types
 - ❑ **Location** – pointer to file location on device
 - ❑ **Size** – current file size
 - ❑ **Protection** – controls who can do reading, writing, executing
 - ❑ **Time, date, and user identification** – data for protection, security, and usage monitoring
-
- ❑ Information about files are kept in the directory structure, which is maintained on the secondary storage
 - ❑ a directory entry consists of the file's name and its unique identifier.
 - ❑ The identifier in turn locates the other file attributes.
 - ❑ It may take more than a kilobyte to record this information for each file.
 - ❑ In a system with many files, the size of the directory itself may be megabytes.
 - ❑ Because directories, like files, must be nonvolatile, they must be stored on the device





File Operations

- File is an **abstract data type**
- **Create**-Two steps –(i)space in the file system must be found for the file.
 - (ii)an entry for the new file must be made in the directory.
- **Write** –make a system call specifying name of the file and the information to be written to the file.
 - system searches the directory to find the file's location.
 - system must keep a *write* pointer to the location in the file where themnext write is to take place.
 - The write pointer must be updated whenever awrite occurs.
- **Read** – use a system call that specifies the name of the file and where (in memory) the next block of the file should be put.
 - directory is searched for the associated entry
 - system needs to keep a *read* pointer to the location in the file where the next read is to take place.
 - Once the read has taken place, the read pointer is updated.
 - the current operation location can be kept as a per-process **currentfile-**
position pointer.





File Operations Contd...

- the current operation location can be kept as a per-process current file-position pointer.
- Both the read and write operations use this same pointer, saving space and reducing system complexity.at read pointer location
- Both the read and write operations use this same pointer, saving space and reducing system complexity.at **read pointer** location
- **Reposition within file** - directory is searched for the appropriate entry
 - current-file-position pointer is repositioned to a given value.
 - Repositioning within a file need not involve any actual I/O.
 - also known as a file **seek**.**seek**
- **Delete** - search the directory for the named file.
 - release all file space, so that it can be reused by other files
 - erase the directory entry
- **Truncate** -user may want to erase the contents of a file but keep its attributes.
- Instead of deleting the file and recreating it, allows all attributes to remain unchanged—except for file length— reset to length zero and its file space released.





- ❑ file operations mentioned involve searching the directory for the entry associated with the named file.
- ❑ To avoid this constant searching, many systems require that an `open()` system call be made before a file is first used actively.
- ❑ The operating system keeps a small table, called the **open-file table**, containing information about all open files.
- ❑ When a file operation is requested, the file is specified via an index into this table, so no searching is required.
- ❑ When the file is no longer being actively used, it is *closed* by the process, and the operating system removes its entry from the open-file table.
- ❑ `create` and `delete` are system calls that work with closed rather than open files.





- ❑ Some systems implicitly open a file when the first reference to it is made.
- ❑ The file is automatically closed when the job or program that opened the file terminates.
- ❑ Most systems, however, require that the programmer open a file explicitly with the `open()` system call before that file can be used.
- ❑ `open()` operation takes a file name and searches the directory, copying the directory entry into the open-file table.
- ❑ The `open()` call can also accept access mode information—create, read-only, read–write, append-only, and so on.
- ❑ This mode is checked against the file’s permissions.
- ❑ If the request mode is allowed, the file is opened for the process.
- ❑ The `open()` system call typically returns a pointer to the entry in the open-file table.
- ❑ This pointer, not the actual file name, is used in all I/O operations, avoiding any further searching and simplifying the system-call interface.





- the operating system uses two levels of internal tables:
 - a **per-process** table and a **system-wide table**.
- The per-process table tracks all files that a process has open.
- stores information regarding the use of the file by the process.
 - the current file pointer for each
 - Access rights to the file
 - accounting information.
 - Each entry in the per-process table in turn points to a system-wide open-file table.
- **The system-wide** table contains process-independent information,
 - the location of the file on disk
 - access dates
 - file size.
- .





- Once a file has been opened by one process, the system-wide table includes an entry for the file
- When another process executes an `open()` call, a new entry is simply added to the process's open-file table pointing to the appropriate entry in the systemwide table.
- the open-file table also has an **open count** associated with each file to indicate how many processes have the file open.
- Each `close()` decreases this *open count*, and when the *open count* reaches zero, the file is no longer in use, and the file's entry is removed from the open-file table





Open Files

- Several pieces of data are needed to manage open files:
 - **Open-file table**: tracks open files
 - **File pointer**: pointer to last read/write location, per process that has the file open
 - **File-open count**: counter of number of times a file is open – to allow removal of data from open-file table when last processes closes it
 - **Disk location of the file**: information needed to locate the file on disk is kept in memory so that the system does not have to read it from disk for each operation.
 - **Access rights**: stored in per-process table -access mode information





Open File Locking

- Provided by some operating systems and file systems
 - Similar to reader-writer locks
 - **Shared lock** similar to reader lock – several processes can acquire concurrently
 - **Exclusive lock** similar to writer lock
- Mediates access to a file
- Mandatory or advisory:
 - **Mandatory** – access is denied depending on locks held and requested
 - **Advisory** – processes can find status of locks and decide what to do





File Types – Name, Extension

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rtf, doc	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	ps, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes compressed, for archiving or storage
multimedia	mpeg, mov, rm, mp3, avi	binary file containing audio or A/V information





File Structure

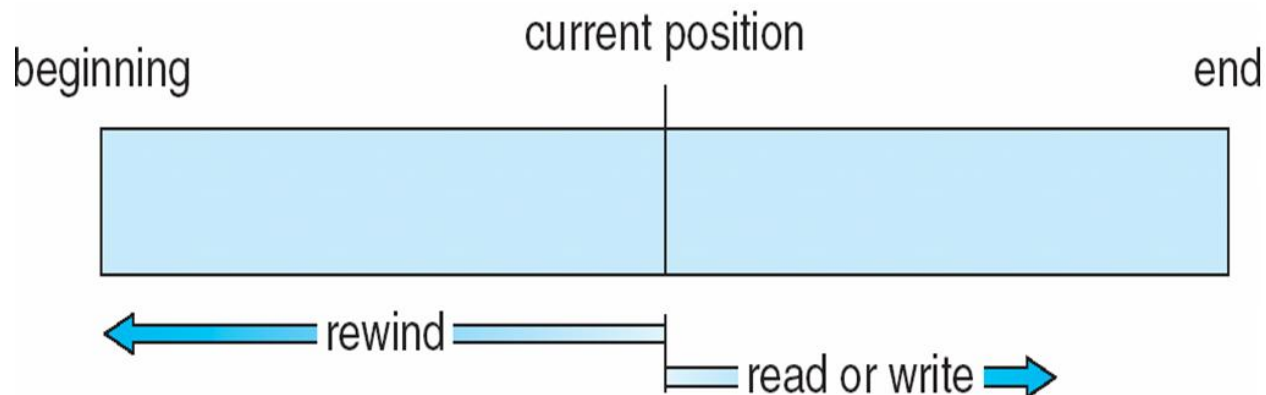
- ❑ File types also can be used to indicate the internal structure of the file
- ❑ certain files must conform to a required structure that is understood by the operating system
- ❑ Some operating systems impose (and support) a minimal number of file structures.
- ❑ considers each file to be a sequence of 8-bit bytes; no interpretation of these bits is made by the operating system.
- ❑ This scheme provides maximum flexibility but little support.
- ❑ Each application program must include its own code to interpret an input file as to the appropriate structure.
- ❑ However, all operating systems must support at least one structure—that of an executable file—so that the system is able to load and run programs.





Access Methods

- **Sequential Access**- simplest access method
 - Information in the file is processed in order, one record after the other.
 - most common; for example, editors and compilers usually access files in this fashion
- **read next** -reads the next portion of the file and automatically advances a file pointer, which tracks the I/O location. Similarly, the write
- **write next** -appends to the end of the file and advances to the end of the newly written material (the new end of file).
- **reset** - Such a file can be reset to the beginning,





Direct-access File

- ❑ **Direct Access** – A file is made up of fixed length **logical records** that allow programs to read and write records rapidly in no particular order.
- ❑ The direct-access method is based on a disk model of a file, since disks allow random access to any file block.
- ❑ For direct access, the file is viewed as a numbered sequence of blocks or records.
- ❑ Thus, we may read block 14, then read block 53, and then write block 7.
- ❑ There are no restrictions on the order of reading or writing for a direct-access file.

```
read n
write n
position to n
    read next
    write next
rewrite n
```

n = relative block number





Simulation of Sequential Access on Direct-access File

sequential access	implementation for direct access
<i>reset</i>	$cp = 0;$
<i>read next</i>	$read\ cp;$ $cp = cp + 1;$
<i>write next</i>	$write\ cp;$ $cp = cp + 1;$





Other Access Methods

- ❑ built on top of a direct-access method.
- ❑ involve the construction of an index for the file.
- ❑ The **index** contains pointers to the various blocks.
- ❑ To find a record in the file, we first search the index and then use the pointer to access the file directly and to find the desired record
- ❑ From this search, we learn exactly which block contains the desired record and access that block.
- ❑ This structure allows to search a large file doing little I/O.
- ❑ With large files, the index file itself may become too large to be kept in memory.
- ❑ One solution is to create an index for the index file.
- ❑ The primary index file would contain pointers to secondary index files, which would point to the actual data items
- ❑ IBM indexed sequential-access method (ISAM)
 - ❑ Small master index, points to disk blocks of secondary index
 - ❑ The secondary index blocks point to the actual file blocks.
 - ❑ File kept sorted on a defined key





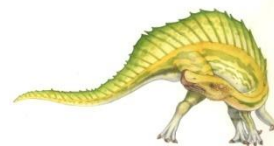
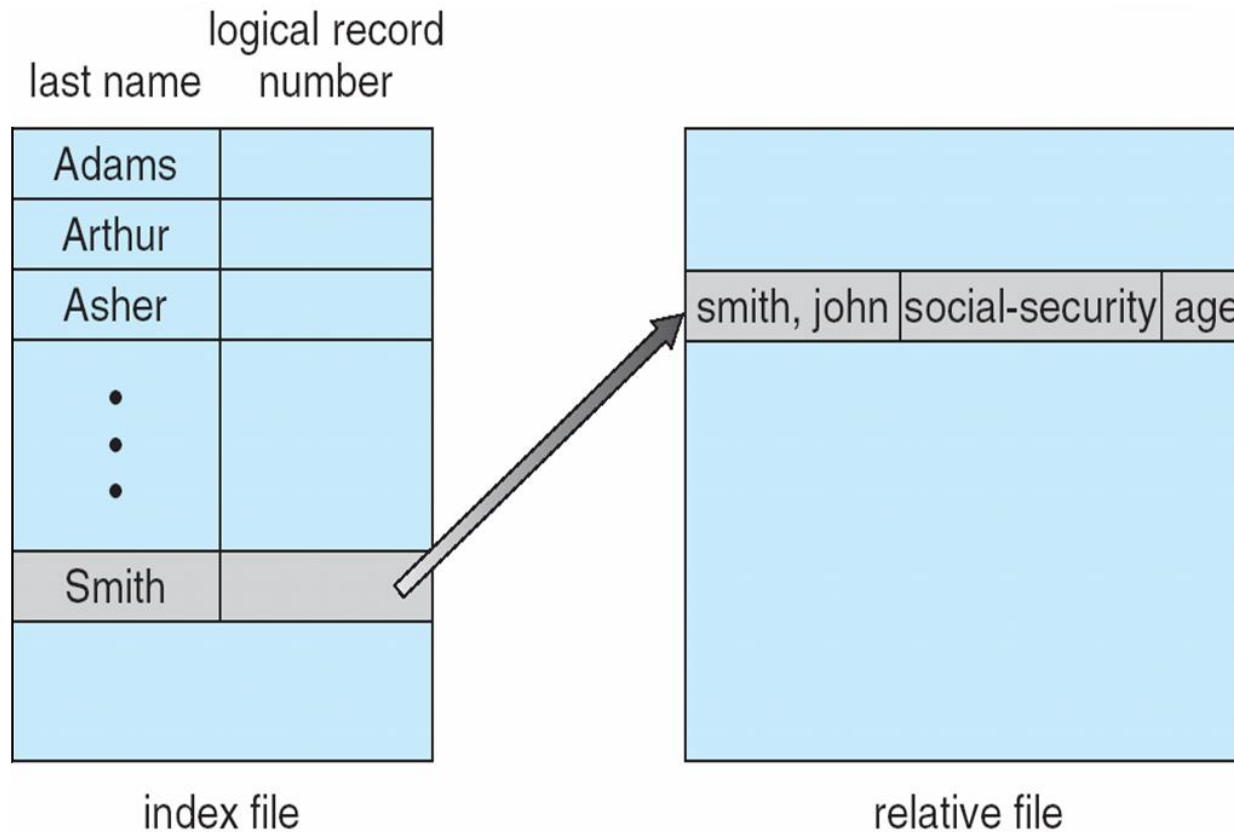
IBM indexed sequential-access method

- IBM indexed sequential-access method (ISAM)
 - Small master index, points to disk blocks of secondary index
 - The secondary index blocks point to the actual file blocks.
 - File kept sorted on a defined key
- To find a particular item, we first make a binary search of the master index, which provides the block number of the secondary index.
- This block is read in, and again a binary search is used to find the block containing the desired record.
- Finally, this block is searched sequentially.
- In this way, any record can be located from its key by at most two direct-access reads





Example of Index and Relative Files





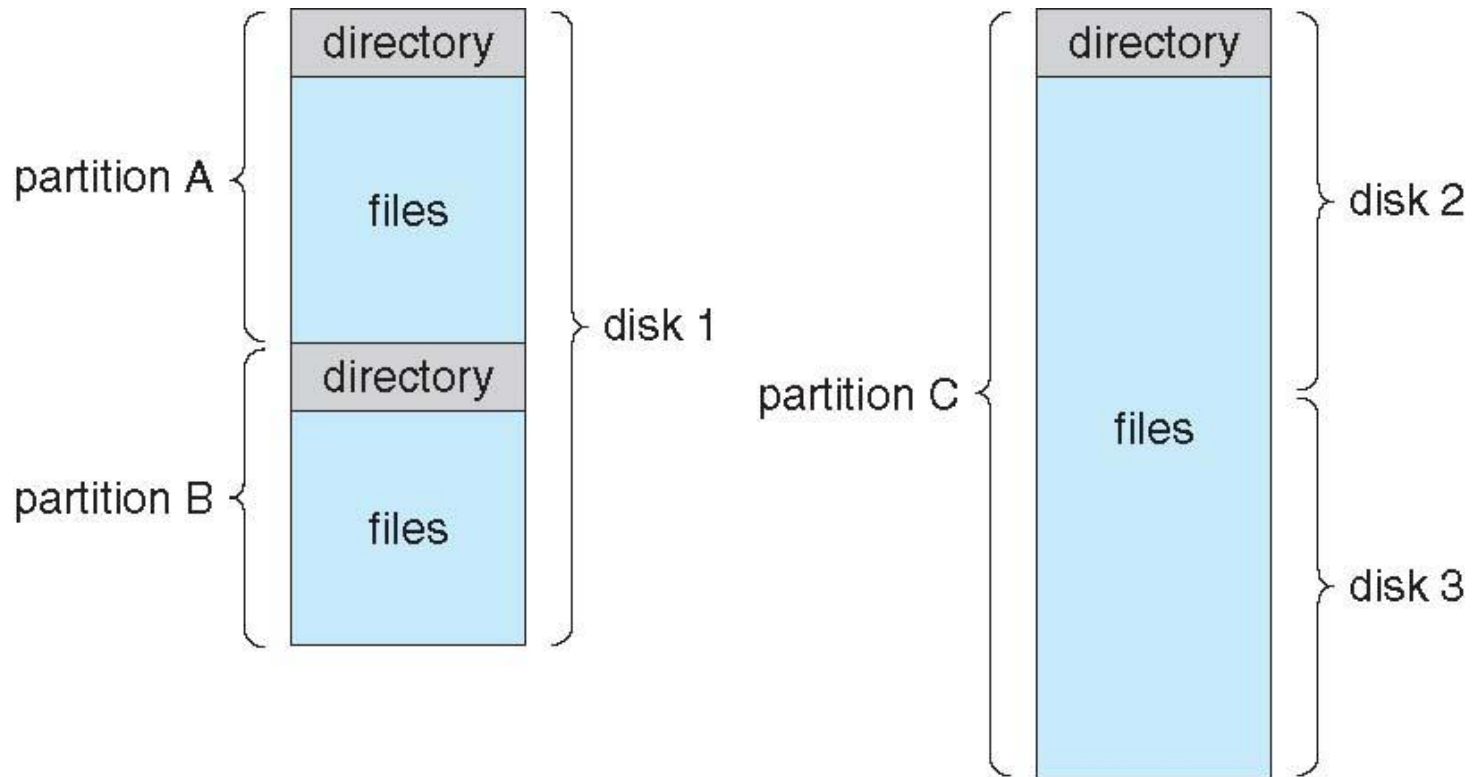
Disk Structure

- ❑ Disk can be subdivided into **partitions**
- ❑ Disks or partitions can be **RAID** protected against failure
- ❑ Disk or partition can be used **raw** – without a file system, or **formatted** with a file system
- ❑ Partitions also known as minidisks, slices
- ❑ Entity containing file system known as a **volume**
- ❑ Each volume containing file system also tracks that file system's info in **device directory** or **volume table of contents**
- ❑ As well as **general-purpose file systems** there are many **special-purpose file systems**, frequently all within the same operating system or computer





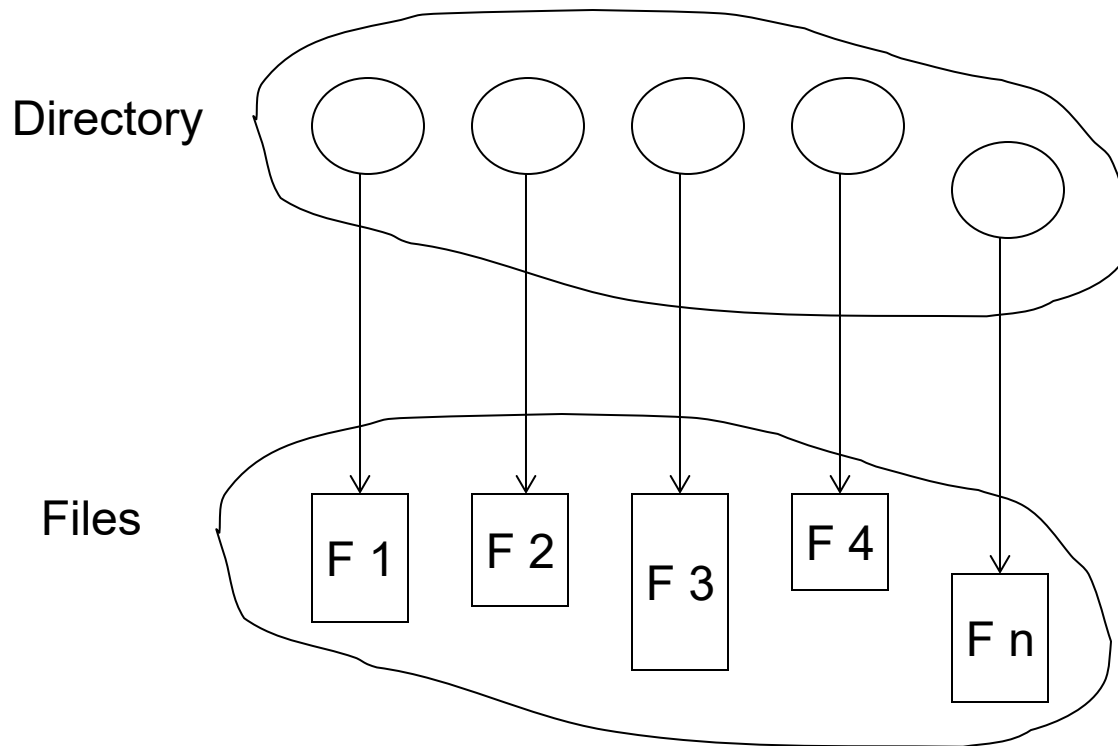
A Typical File-system Organization





Directory Structure

- A collection of nodes containing information about all files



Both the directory structure and the files reside on disk





Types of File Systems

- We mostly talk of general-purpose file systems
- But systems frequently have many file systems, some general- and some special- purpose
- Consider Solaris has
 - tmpfs – memory-based volatile FS for fast, temporary I/O
 - objfs – interface into kernel memory to get kernel symbols for debugging
 - ctfs – contract file system for managing daemons
 - lofs – loopback file system allows one FS to be accessed in place of another
 - procfs – kernel interface to process structures
 - ufs, zfs – general purpose file systems





Operations Performed on Directory

- Search for a file
- Create a file
- Delete a file
- List a directory
- Rename a file
- Traverse the file system





Directory Organization

The directory is organized logically to obtain

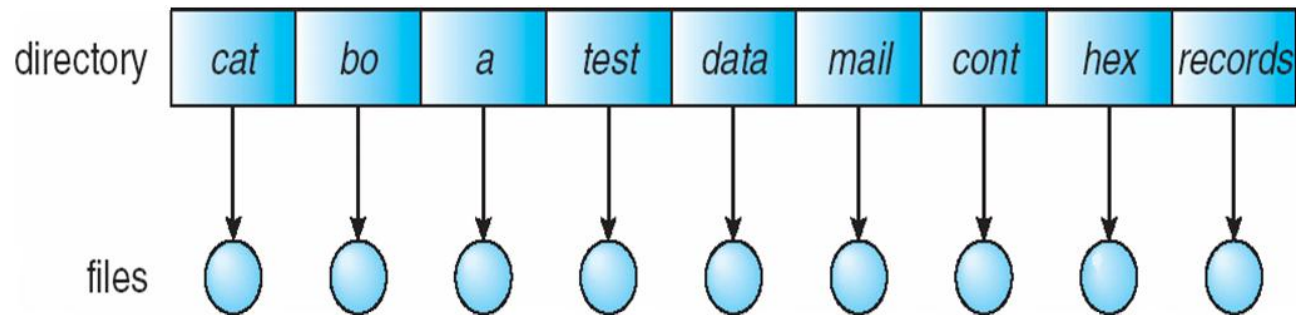
- Efficiency – locating a file quickly
- Naming – convenient to users
 - Two users can have same name for different files
 - The same file can have several different names
- Grouping – logical grouping of files by properties, (e.g., all Java programs, all games, ...)





Single-Level Directory

- A single directory for all users



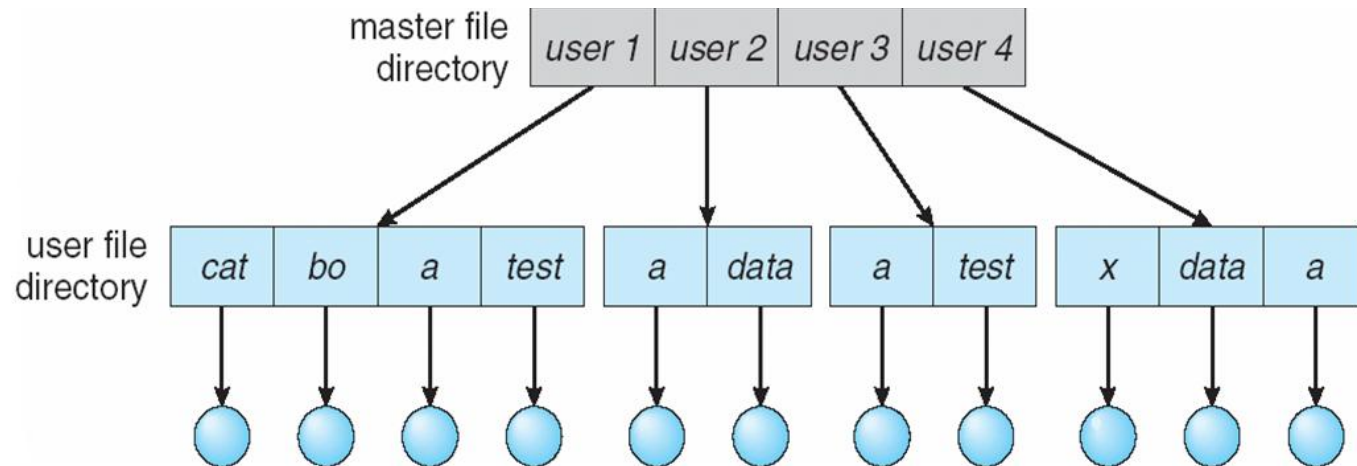
- Naming problem
- Grouping problem





Two-Level Directory

- ❑ Separate directory for each user

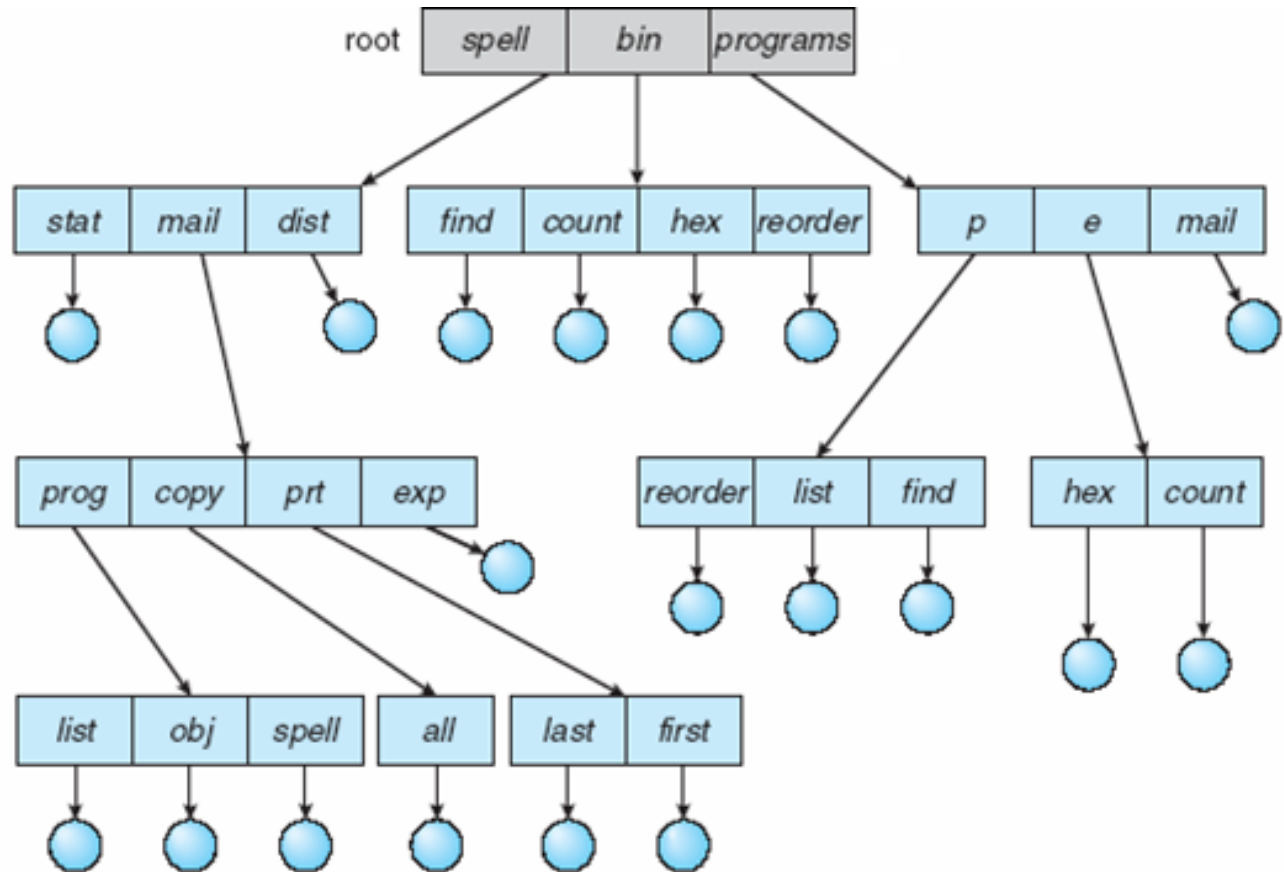


- ❑ Solves name-collision problem,
- ❑ user name and a file name define a *path name*
- ❑ Can have the same file name for different user
- ❑ Efficient searching
- ❑ No grouping capability





Tree-Structured Directories





Tree-Structured Directories (Cont.)

- ❑ Efficient searching
- ❑ Grouping Capability
- ❑ Current directory (working directory)
- ❑ Path names can be of two types: *absolute* and *relative*.
- ❑ An **absolute path name** begins at the root and follows a path down to the specified file, giving the directory names on the path.
- ❑ A **relative path name** defines a path from the current directory.
- ❑ If the current directory is *root/spell/mail*, then the relative path name *prt/first*
- ❑ refers to the same file as does the absolute path name *root/spell/mail/prt/first*





Tree-Structured Directories (Cont)

- ❑ **Absolute** or **relative** path name
- ❑ Creating a new file is done in current directory
- ❑ Delete a file

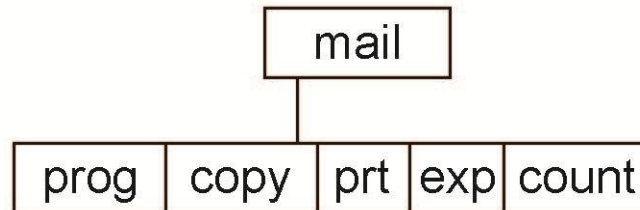
rm <file-name>

- ❑ Creating a new subdirectory is done in current directory

mkdir <dir-name>

Example: if in current directory **/mail**

mkdir count



Deleting “mail” $\Rightarrow$ deleting the entire subtree rooted by “mail”





Deletion of a directory

- ❑ If a directory is empty, its entry in the directory that contains it can simply be deleted.
- ❑ suppose the directory to be deleted is not empty but contains several files or subdirectories. One of two approaches can be taken
- ❑ to delete a directory, the user must first delete all the files in that directory.
- ❑ If any subdirectories exist, this procedure must be applied recursively to them, so that they can be deleted also.
- ❑ This approach can result in a substantial amount of work.
- ❑ An alternative approach, such as that taken by the UNIX `rm` command, is to provide an option: when a request is made to delete a directory, all that directory's files and subdirectories are also to be deleted.
- ❑ Either approach is fairly easy to implement;
- ❑ The latter policy is more convenient, but it is also more dangerous, because an entire directory structure can be removed with one command.





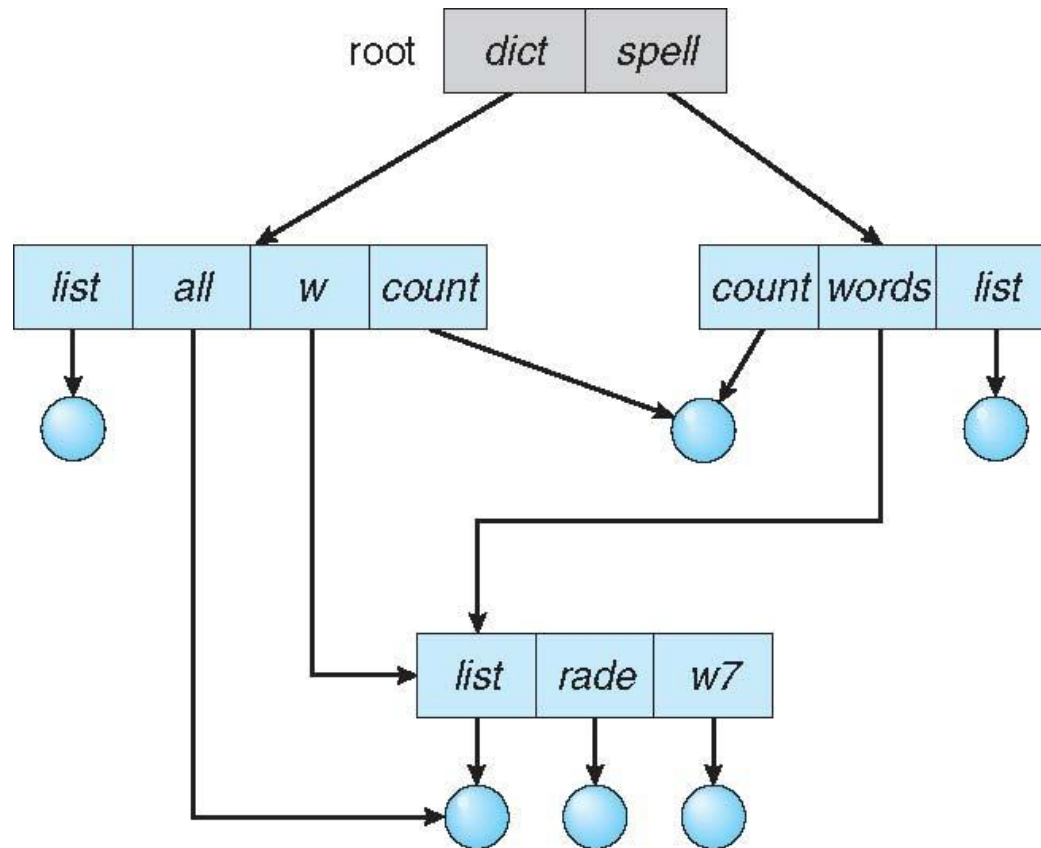
- With a tree-structured directory system, users can be allowed to access, in addition to their files, the files of other users.
- For example, user B can access a file of user A by specifying its path names.
- User B can specify either an absolute or a relative path name.
- Alternatively, user B can change her current directory to be user A's directory and access the file by its file names.





Acyclic-Graph Directories

- allows directories to share subdirectories and files





Acyclic-Graph Directories (Cont.)

- With a shared file, only *one* actual file exists, so any changes made by one user are immediately visible to the other.
- Sharing is particularly important for subdirectories; a new file created by one person will automatically appear in all the shared subdirectories
- Two different names (aliasing)
- If **dict** deletes **list** $\Rightarrow$ dangling pointer

Solutions:

- Backpointers, so we can delete all pointers
Variable size records a problem
- Backpointers using a daisy chain organization
- Entry-hold-count solution
- New directory entry type
 - **Link** – another name (pointer) to an existing file
 - **Resolve the link** – follow pointer to locate the file





Implementation of shared files and subdirectories

- ❑ Shared files and subdirectories can be implemented in several ways.
- ❑ A common way, exemplified by many of the UNIX systems, is to create a new directory entry called a link.
- ❑ A link is effectively a pointer to another file or subdirectory.
- ❑ For example, a link may be implemented as an absolute or a relative path name.
- ❑ When a reference to a file is made, we search the directory.
- ❑ If the directory entry is marked as a link, then the name of the real file is included in the link information.
- ❑ We resolve the link by using that path name to locate the real file.
- ❑ Links are easily identified by their format in the directory entry and are effectively indirect pointers.
- ❑ The operating system ignores these links when traversing directory trees to preserve the acyclic structure of the system.





Implementation of shared files and subdirectories Contd..

- ❑ Another common approach to implementing shared files is to duplicate all information about them in both sharing directories.
- ❑ Both entries are identical and equal.
- ❑ Consider the difference between this approach and the creation of a link.
- ❑ The link is clearly different from the original directory entry; thus, the two are not equal.
- ❑ Duplicate directory entries, however, make the original and the copy indistinguishable.
- ❑ A major problem with duplicate directory entries is maintaining consistency when a file is modified





- When can the space allocated to a shared file be deallocated and reused?
- One possibility is to remove the file whenever anyone deletes it, but this action may leave dangling pointers to the now-nonexistent file.
- Worse, if the remaining file pointers contain actual disk addresses, and the space is subsequently reused for other files, these dangling pointers may point into the middle of other files.
- In a system where sharing is implemented by symbolic links
 - The deletion of a link need not affect the original file; only the link is removed.
 - If the file entry itself is deleted, the space for the file is deallocated, leaving the links dangling.
 - We can search for these links and remove them as well, but unless a list of the associated links is kept with each file, this search can be expensive.
- .





- ❑ Alternatively, we can leave the links until an attempt is made to use them.
- ❑ At that time, we can determine that the file of the name given by the link does not exist and can fail to resolve the link name; the access is treated just as for any other illegal file name.
- ❑ In the case of UNIX, symbolic links are left when a file is deleted, and it is up to the user to realize that the original file is gone or has been replaced.
- ❑ Microsoft Windows uses the same approach
- ❑ Another approach to deletion is to preserve the file until all references to it are deleted.
- ❑ To implement this approach, must have some mechanism for determining that the last reference to the file has been deleted.
- ❑ keep a list of all references to a file (directory entries or symbolic links).
- ❑ When a link or a copy of the directory entry is established, a new entry is added to the file-reference list.
- ❑ When a link or directory entry is deleted, we remove its entry on the list.
- ❑ The file is deleted when its file-reference list is empty



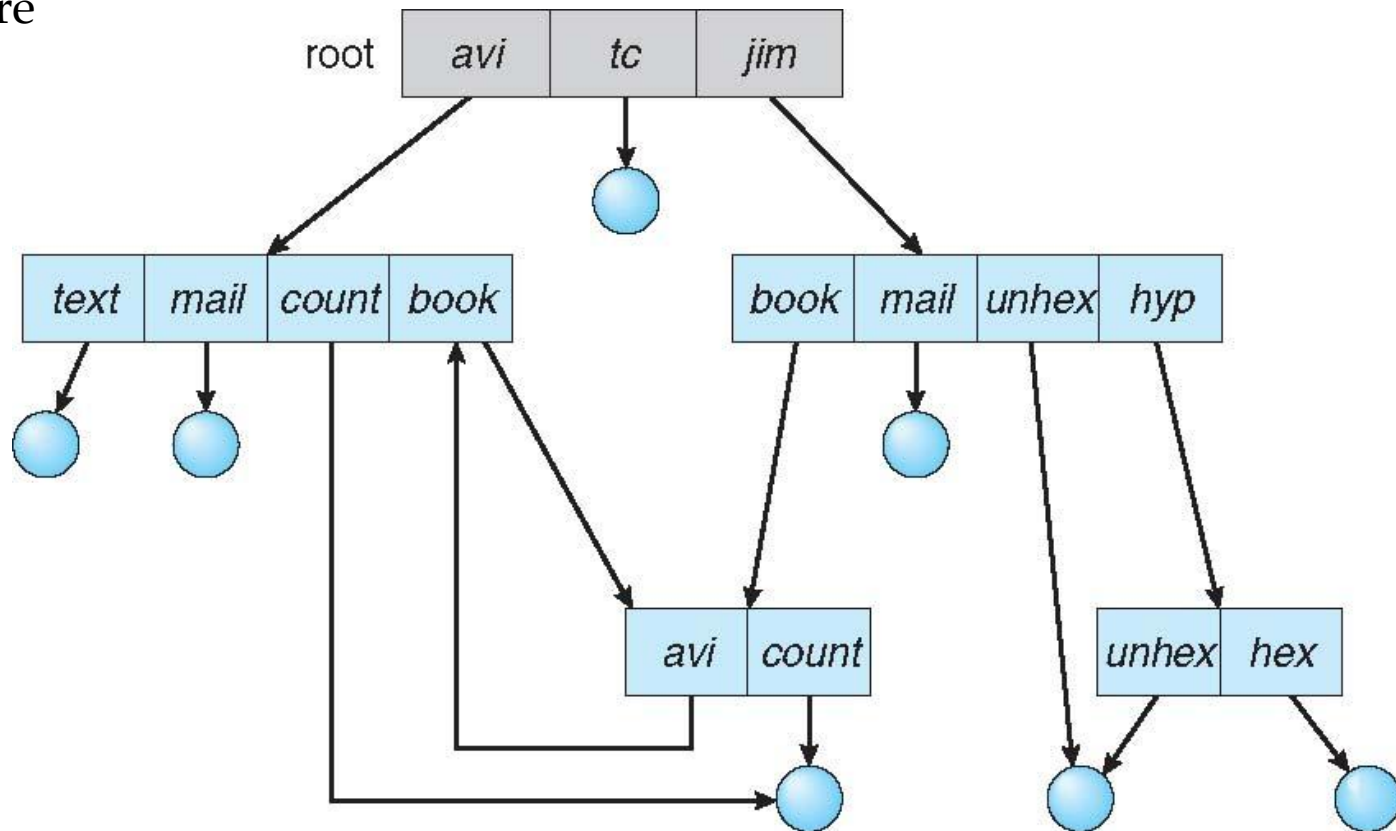


General Graph Directory

If we start with a two-level directory and allow users to create subdirectories, a tree-structured directory results.

simply adding new files and subdirectories to an existing tree-structured directory preserves the tree-structured nature.

when we add links, the tree structure is destroyed, resulting in a simple graph structure





General Graph Directory (Cont.)

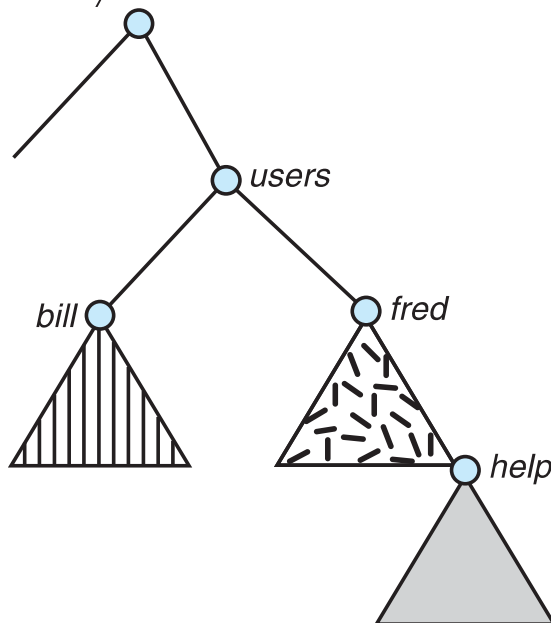
- How do we guarantee no cycles?
 - Allow only links to file not subdirectories
 - **Garbage collection**
 - Every time a new link is added use a cycle detection algorithm to determine whether it is OK



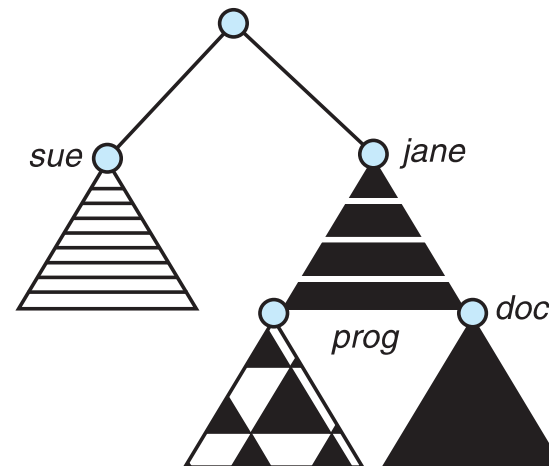


File System Mounting

- A file system must be **mounted** before it can be accessed
- A unmounted file system is mounted at a **mount point**
- The operating system is given the name of the device and the **mount point**—the location within the file structure where the file system is to be attached.



(a)



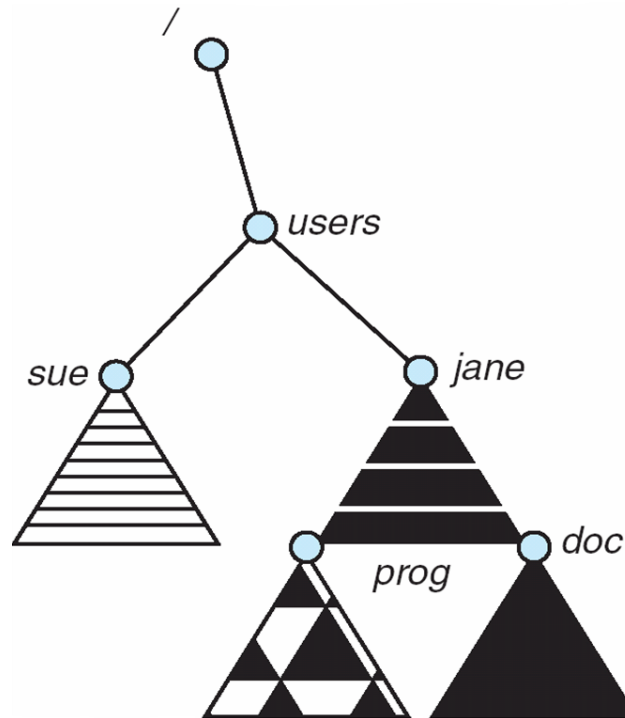
(b)

File system. (a) Existing system. (b) Unmounted volume

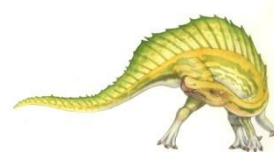




Mount Point



Mount point





Reference

- A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, (9e), Wiley and Sons (Asia) Pvt. Ltd, 2016.

